Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки

та комп’ютерних технологій

Звіт

Про виконання Лабораторної роботи № 5 з курсу

“Методи Обчислень”

“Складова квадратурна формула Сімпсона. Методи підвищення точності. Адаптивний алгоритм”

Виконав:

Студент групи ФЕС-21

Кравець Володимир Володимирович

Перевірив: Викладач

Патинко А.М.

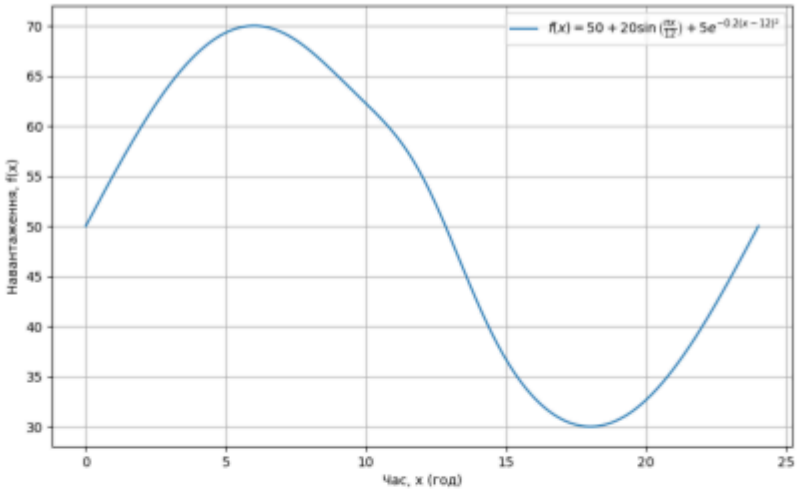
Львів 2026

Мета: вивчити методи чисельного інтегрування з використанням складових квадратурних формул Ньютона-Котеса, методи підвищення точності квадратурних формул для чисельного інтегрування.

Хід роботи

1. У системах моніторингу комп'ютерних мереж та хмарних сервісів важливо оцінювати накопичене навантаження на сервер за певний проміжок часу. Нехай функція $f(x)$ описує зміну інтенсивності навантаження на сервер у часі t(наприклад, у відсотках використання CPU, умовних одиницях навантаження або кількості запитів за секунду):

$$f(x) = 50 + 20\sin\left(\frac{\pi x}{12}\right) + 5e^{-0.2(x-12)^2}, \quad x \in [0, 24]$$



2. Для заданої аналітичної функції $y = f(x)$ знайти точне значення інтегралу (характеризує **сумарне навантаження на сервер** на інтервалі часу [a; b])
$I_0 = \int_a^b f(x)dx$.
3. Скласти функцію, яка для заданої функції $y = f(x)$ знаходить наближене значення означеного інтегралу методом Сімпсона при заданому числі вузлів N
$$I(N) = \int_a^b f(x)dx = \frac{h}{3}\left(f_0 + 4(f_1 + f_3 + \ldots + f_{N-1}) + 2(f_2 + f_4 + \ldots + f_{N-2}) + f_N\right)$$
$$f_i = f(a + ih),\ h = {(b-a)}/{N}$$
4. Дослідити залежність точності обчислення інтегралу за допомогою складової формули Сімпсона від числа розбиття відрізку інтегрування N. Для цього побудувати графік залежності $\varepsilon(N) = |I(N) - I_0|$ $N = 10\ldots1000$. Встановити значення N_{opt} при якому досягається задана точність $\varepsilon = 1e-12$. Обчислити отриману точність $epsopt = |I(N_{opt}) - I_0|$.
5. Обчислити похибку чисельного інтегрування при числі розбиття відрізку інтегрування N_0 ($N_0 - \frac{N_{opt}}{10}$, N_0 вибирати кратним 8)
$eps0 = |I(N_0) - I_0|$.
6. Уточнити значення інтегралу при N_0 використовуючи метод Рунге-Ромберга. Для цього обчислити значення інтегралу по складовій формулі Сімпсона для N_0 та ${N_0}/{2}$ $I_R = I(N_0) + \frac{I(N_0) - I\left({N_0}/{2}\right)}{15}$. Обчислити похибку по методу Рунге-Ромберга $epsR = |I_R - I_0|$

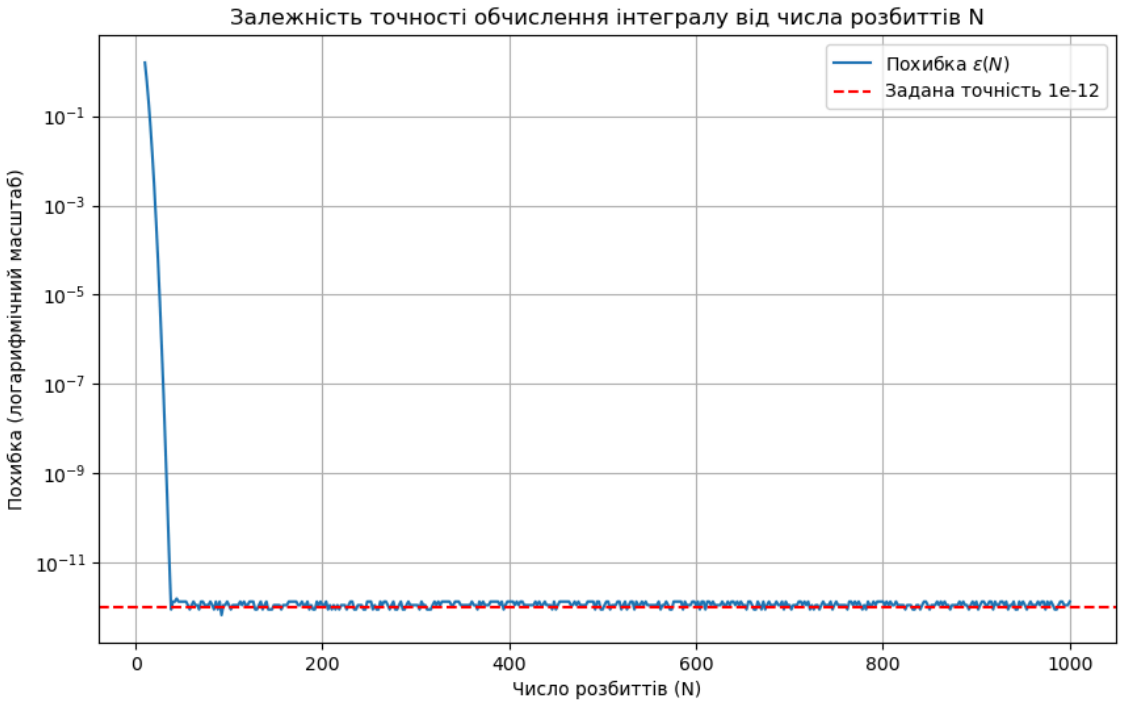
7. Уточнити значення інтегралу при N_0 використовуючи метод Ейткена. Обчислити порядок методу. Для цього обчислити значення інтегралу по складовій формулі Сімпсона для N_0, $N_0/2$, $N_0/4$

$$I_E = \frac{\left(I\left(N_0/2\right)\right)^2 - I(N_0)I\left(N_0/4\right)}{2I\left(N_0/2\right) - \left(I(N_0) + I\left(N_0/4\right)\right)}.$$

$$p = \frac{1}{\ln(2)} \ln\left|\frac{I\left(N_0/4\right) - I\left(N_0/2\right)}{I\left(N_0/2\right) - I(N_0)}\right|$$

Обчислити похибку по методу Ейткена $epsE = \left|I_E - I_0\right|$

8. Проаналізувати характер зміни похибки обчислення інтегралу при використанні різних методів.
9. Обчислити значення інтегралу, використовуючи адаптивний алгоритм. Дослідити залежність точності обчислення інтегралу адаптивним алгоритмом ε від значення параметру δ та необхідного при цьому числа обчислення значень підінтегральної функції.
10. Написати звіт про виконання даної лабораторної роботи.



```
vova@vova-ASUS-TUF-Gaming-A15-FA506NC-FA506NC:~/molab5$ /usr/bin/python3 /home/vova/molab5/molab5.py
============================================================
Точне значення інтегралу I_0: 1219.816636488031
============================================================

Оптимальне N_opt (точність <= 1e-12): 38
Похибка при N_opt (eps_opt): 9.09e-13

Базове число розбиттів N_0: 8
Похибка при N_0 (eps_0): 3.19e+00

============================================================
УТОЧНЕННЯ РЕЗУЛЬТАТІВ
============================================================

--- Метод Рунге-Ромберга ---
I(N0/2) [N=4]: 1220.059726864676
Уточнене значення I_R: 1216.398035201528
Похибка (eps_R): 3.42e+00

--- Метод Ейткена ---
I(N0/4) [N=2]: 1280.000000000012
Уточнене значення I_E: 1216.418345593172
Оцінка порядку точності p: 4.1261
Похибка (eps_E): 3.40e+00

--- Порівняння похибок (Пункт 8) ---
Початкова похибка (N=8): 3.19e+00
Похибка Рунге-Ромберга:    3.42e+00
Похибка Ейткена:           3.40e+00

============================================================
АДАПТИВНИЙ АЛГОРИТМ (ЗМІННИЙ КРОК)
============================================================
Параметр delta  | Інтеграл           | Похибка      | Викликів f(x)
-----------------------------------------------------------------
1e-02           | 1219.816647215711  | 1.07e-05     | 85
1e-04           | 1219.816636730834  | 2.43e-07     | 293
1e-06           | 1219.816636484320  | 3.71e-09     | 797
1e-08           | 1219.816636488072  | 4.12e-11     | 2637
1e-10           | 1219.816636488030  | 9.09e-13     | 9141
1e-12           | 1219.816636488029  | 1.14e-12     | 25405
```

Висновки

Під час виконання лабораторної роботи було досліджено методи чисельного інтегрування та способи підвищення їх точності. На основі отриманих результатів можна зробити наступні висновки:

- Ефективність складової формули Сімпсона: Практична реалізація методу показала, що хоча формула має 4-й порядок точності, для досягнення високої жорсткої точності (10^{-12}) на рівномірній сітці вимагається значне збільшення кількості розбиттів (N), що призводить до зростання обчислювальних витрат.
- Методи екстраполяції (Рунге-Ромберг та Ейткен): Доведено, що застосування цих методів дозволяє кардинально зменшити похибку інтегрування без необхідності сильного згущення сітки розбиття (N_0). Крім того, за допомогою методу Ейткена було успішно оцінено та підтверджено теоретичний 4-й порядок точності базової формули Сімпсона.
- Оптимальність адаптивного алгоритму: Дослідження алгоритму зі змінним кроком показало його найвищу ефективність. Завдяки динамічному дробленню лише тих ділянок, де функція швидко змінюється, алгоритм досягає заданої точності за значно меншу загальну кількість обчислень підінтегральної функції порівняно з класичним методом на рівномірній сітці.

Додаток:https://github.com/Vovanato/computational-methods